23CSE455 Design Patterns
Case Study Report

Team: B14
Case study: Design Pattern Identification and Evolution in a Next.js-Based Digital News and Publishing Application
Application: Next.js Blog CMS
Date: 2026-10-28 (Part 3 records corrected)
Submission repository: 23CSE455_DP_CaseStudy

Students

Roll number	Student name	Batch
AM.SC.U4AIE23121	Keziah Suzaine George	AIE
AM.SC.U4AIE23139	Nivedita Ramesh	AIE
AM.SC.U4CSE23360	Soorya Krishna	CSE
AM.SC.U4CSE23309	B R Govind	CSE

1 Application and architecture

Original source. Repository: `https://github.com/thepracticalengineer/nextjs-blog-cms.git`. Baseline commit: `90ec6565877053ee0e93e9f09dcc7c7213aa4832` (merge of pull request #102, 8 October 2026). Access date: 2026-10-10. License: MIT.

Application. A full-stack blog CMS with a TipTap editor, Supabase authentication including MFA, Admin and Author roles, a draft and publication workflow, comments, tags and categories, public author profiles, newsletter subscriptions, an AI writing assistant (Claude, Gemini, OpenAI) and a developer REST API.

Environment.	Node.js	24.14.0 (repository requests 24.21.0; mismatch noted, nothing failed)
	Package manager	pnpm 10.29.3
	Framework	Next.js 16.3.8 (App Router), React 19.3.0, TypeScript 6.0.3
	Database and auth	Supabase (supabase-js 2.117.2, @supabase/ssr 0.12.7)
	Other	Zustand 5.0.15, Zod 4.6.5, Vitest 5.0.3, Playwright 1.63.0

Scope. Inspected: `app/`, `components/`, `features/`, `lib/` and `proxy.ts`. Excluded: `node_modules`, `.next`, `.git`, `.husky`, `.github`, `.agents`, `.claude`, `docs/`, `public/`, `database/`, `supabase/migrations/`, `e2e/`, `__tests__/`, lockfiles and generated files. Reason: not application logic or not relevant to pattern discovery. Unit tests were used only as supporting evidence. The source copy in `code/` equals the baseline commit minus `.husky`, `.github`, `tsconfig.tsbuildinfo` and `package-lock.json`.

Architecture.

The browser reaches public pages, dashboard pages, authentication pages and API routes. Dashboard and authentication paths pass through `proxy.ts` first; its `matcher` covers only `/dashboard`, `/login`, `/register`, `/mfa`, `/forgot-password` and `/reset-password`, so public pages do not. Pages and routes call domain modules in `features/`, which use the Supabase client creators in `lib/supabase`, the permission table in `lib/permissions`, the LLM service and the notification modules.

Request flow (admin opens the Users page). The browser requests `/dashboard/admin/users`. Next.js first runs `proxy.ts`, which refreshes the Supabase session with `supabase.auth.getUser()` (line 34), redirects unauthenticated users to `/login`, sends users with an enrolled but unverified second factor to `/mfa`, and, for paths under `/dashboard/admin`, reads the user's role from the `profiles` table and redirects non-admins to `/dashboard`. When the request passes, the server component `UsersPage` (`app/(dashboard)/dashboard/admin/users/page.tsx`) runs. It

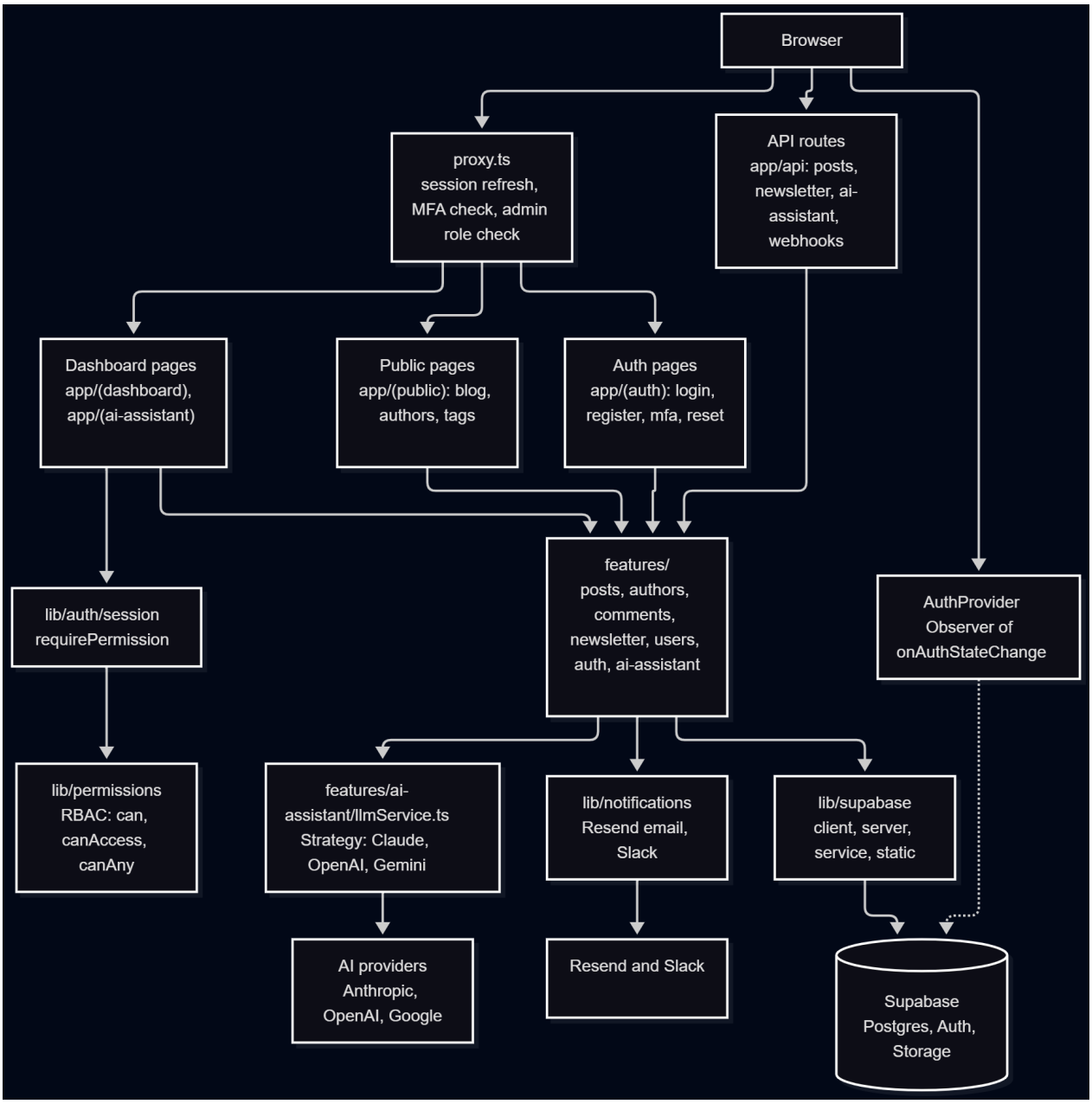


Figure 1: Architecture of the application at the baseline commit.

calls `requirePermission('users:read')` (line 9), loads data with `getAllUsers()` from `features/users/queries` (line 11) and renders `UserTable` (line 21). In the browser, `AuthProvider` keeps session state current by observing `supabase.auth.onAuthStateChange`.

Run instructions. See `README.md`. Results at the baseline:

Command	Result
`pnpminstall--frozen-lockfile`	Succeeded, 933 packages
`pnpmruntest:run`	927 of 927 passed in 83 files on default settings (runs of 71.43 s and 80.10 s). Log: `test_result/test_result_vitest.log`
`pnpmexecvitestrun--testTimeout=30000--maxWorkers=2`	83 test files passed, 927 of 927 tests passed (222.58 s). Screenshot: `test_result/vitest_summary.png`
`pnpmruntypecheck`	Passed. Log: `test_result/typecheck.log`; screenshot: `test_result/typecheck.png`
`pnpmrundev`, `pnpmruntest:e2e`	Not run: Docker is not installed, so local Supabase was unavailable

Two further default-setting runs under heavy machine load failed with 6 and 12 failures (130 s and 188 s). Every failure was a 5000 ms timeout in a jsdom UI test, or a follow-on assertion in the same file after a timeout. The repository sits in a OneDrive-synced folder. With a 30 s timeout and 2 workers the same code passed 927 of 927, so the failures were timing-related and not application defects.

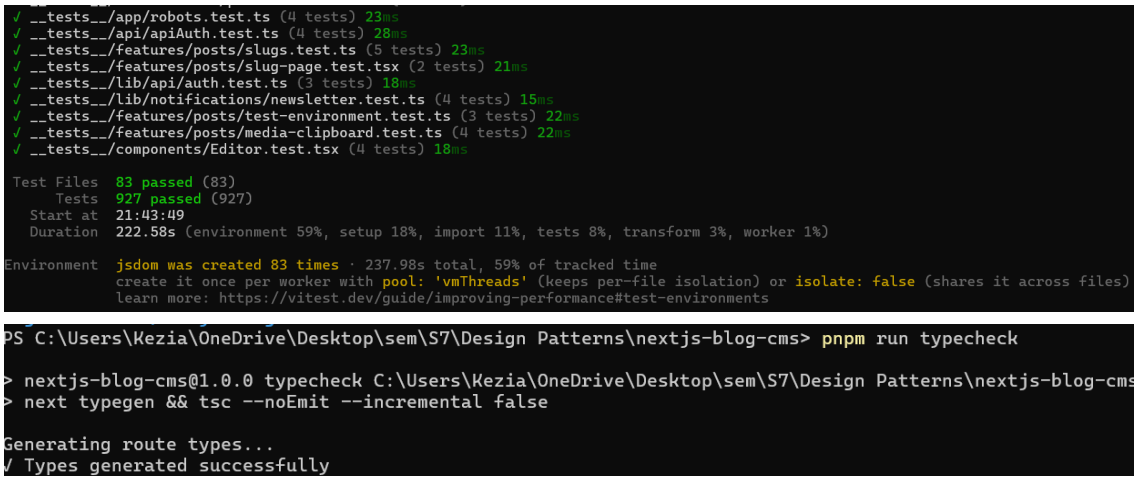


Figure 2: Passing unit-test run and typecheck at the baseline commit.

2 Part 1: Pattern discovery

Method. Candidates were found by reading the file listing and then reading the likely files in full with line numbers (saved in `Part1/evidence/`). Each candidate was checked against three questions: what problem it solves here, whether every role maps to a real symbol, and whether the participants interact as the pattern defines. A class name, a file name or an LLM statement alone was not accepted as evidence. Combined or absent roles are stated in the records; no classes were invented.

2.1 Totals at the baseline

Confirmed pattern types	3 (Strategy, Observer, Role-Based Access Control)
Confirmed pattern instances	3
Concrete participant classes	7, all functions or components rather than classes: `streamClaude`, `streamOpenAI`, `streamGemini`; `AuthProvider`; `can`, `canAccess`, `canAny`
Unique participant symbols	15 application symbols, plus 1 from a dependency (`supabase.auth.onAuthStateChange`)
Rejected candidates	7
Uncertain candidates	1
Observation scope	Files listed in `Part1/rejected_uncertain.md`, at the baseline commit

2.2 Pattern inventory

Pattern type	Instance name	Instances	Concrete classes
Strategy	LLM Provider Streaming Strategy	1	3
Observer	Auth State Observer	1	1
RBAC	RBAC Permission Table	1	3

2.3 Records for each pattern instance

Instance 1: LLM Provider Streaming Strategy

Pattern type and category: Strategy, GoF behavioral. **Purpose and origin:** streams chat replies from Claude, OpenAI or Gemini through one call; application code.

Role	Symbol	File and lines
Strategy	Not separately declared (implicit signature `(params: SendMessageParams)=> AsyncGenerator<string>`)	`features/ai-assistant/llmService.ts`:10–16 (params type)
ConcreteStrategy	`streamClaude`	same file:69–90
ConcreteStrategy	`streamOpenAI`	same file:94–118
ConcreteStrategy	`streamGemini`	same file:129–146
Context	`sendMessage`	same file:150–155

Evidence. The three functions share one parameter type and return type. The context selects one at run time from `params.provider` and delegates:

```
export function sendMessage(params: SendMessageParams): AsyncGenerator<string> {
  if (params.provider === 'claude') return streamClaude(params)
  if (params.provider === 'openai') return streamOpenAI(params)
  if (params.provider === 'gemini') return streamGemini(params)
  throw new Error(`Unsupported provider: ${params.provider}`)
}
```

Limitation. There is no declared interface, and selection is a hard-coded `if` chain rather than injection or a registry.

Verification: Confirmed. Reviewers: team B14. Method: source reading of the listed ranges; `__tests__/ai-assistant/llmService.test.ts` (20 tests) passes in the baseline run but was not

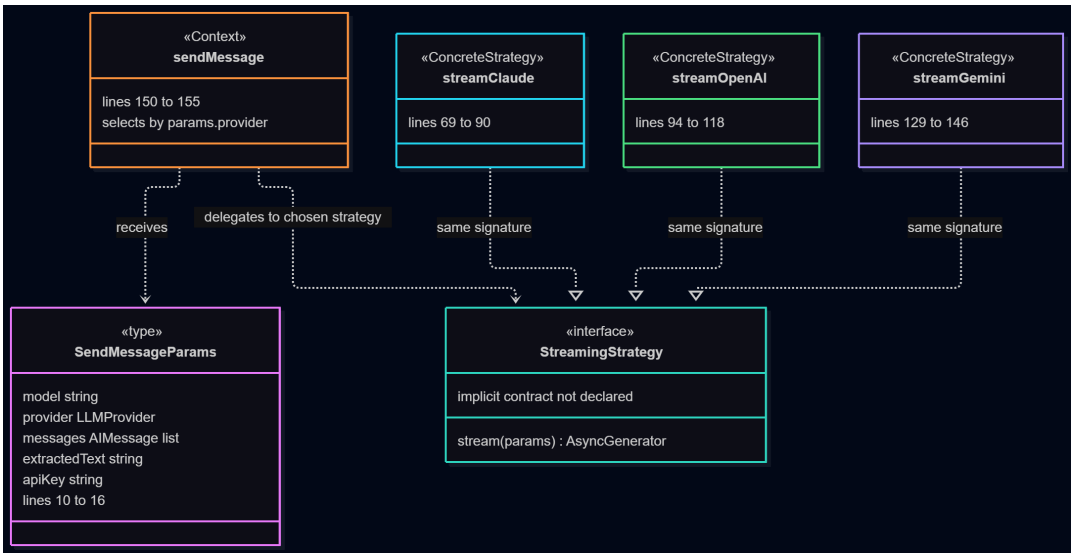


Figure 3: UML for the Strategy instance (source: `Part1/uml_strategy.mmd`).

the sole evidence. Alternative considered: the repeated provider branches in `generateBlogPost`, `generateBlogPostHeadless`, `generateChatTitle` and `validateProviderKey` (lines 157–386) do not apply Strategy; see the rejected list.

Instance 2: Auth State Observer

Pattern type and category: Observer, GoF behavioral. **Purpose and origin:** keeps UI session and profile state in sync with login, logout and token refresh; application code using the Supabase dependency.

Role	Symbol	File and lines
Subject	`supabase.auth` (dependency, not application code)	`features/auth/context/AuthProvider.tsx`:37–39
ConcreteObserver	`AuthProvider`, via the callback given to `onAuthStateChange`	same file:9–59 (callback 39–48)
Detach	`subscription.unsubscribe()`	same file:50
State holder	`AuthContext`, `AuthContext.Provider`	`AuthContext.tsx`:14–19; `AuthProvider.tsx`:54–58
Consumers	`useAuth`; `useSession`	`AuthContext.tsx`:21–23; `features/auth/hooks/useSession.ts`:5–8

Evidence. The component subscribes, updates state when the callback fires, and unsubscribes in the effect cleanup:

```
const { data: { subscription } } = supabase.auth.onAuthStateChange(
  async (_event, session) => { setSession(session); setUser(session?.user ?? null); ... })
return () => subscription.unsubscribe()
```

The subject belongs to `@supabase/supabase-js`, so no subject class is invented. React Context is the delivery mechanism to consumers and is not counted as a second instance.

Verification: Confirmed. Reviewers: team B14. Method: source reading of `AuthProvider.tsx`, `AuthContext.tsx` and `useSession.ts`.

Instance 3: RBAC Permission Table

Pattern type and category: Role-Based Access Control, architecture and security pattern (not a GoF pattern). **Purpose and origin:** centralizes which role may perform which action; application code.

Role	Symbol	File and lines

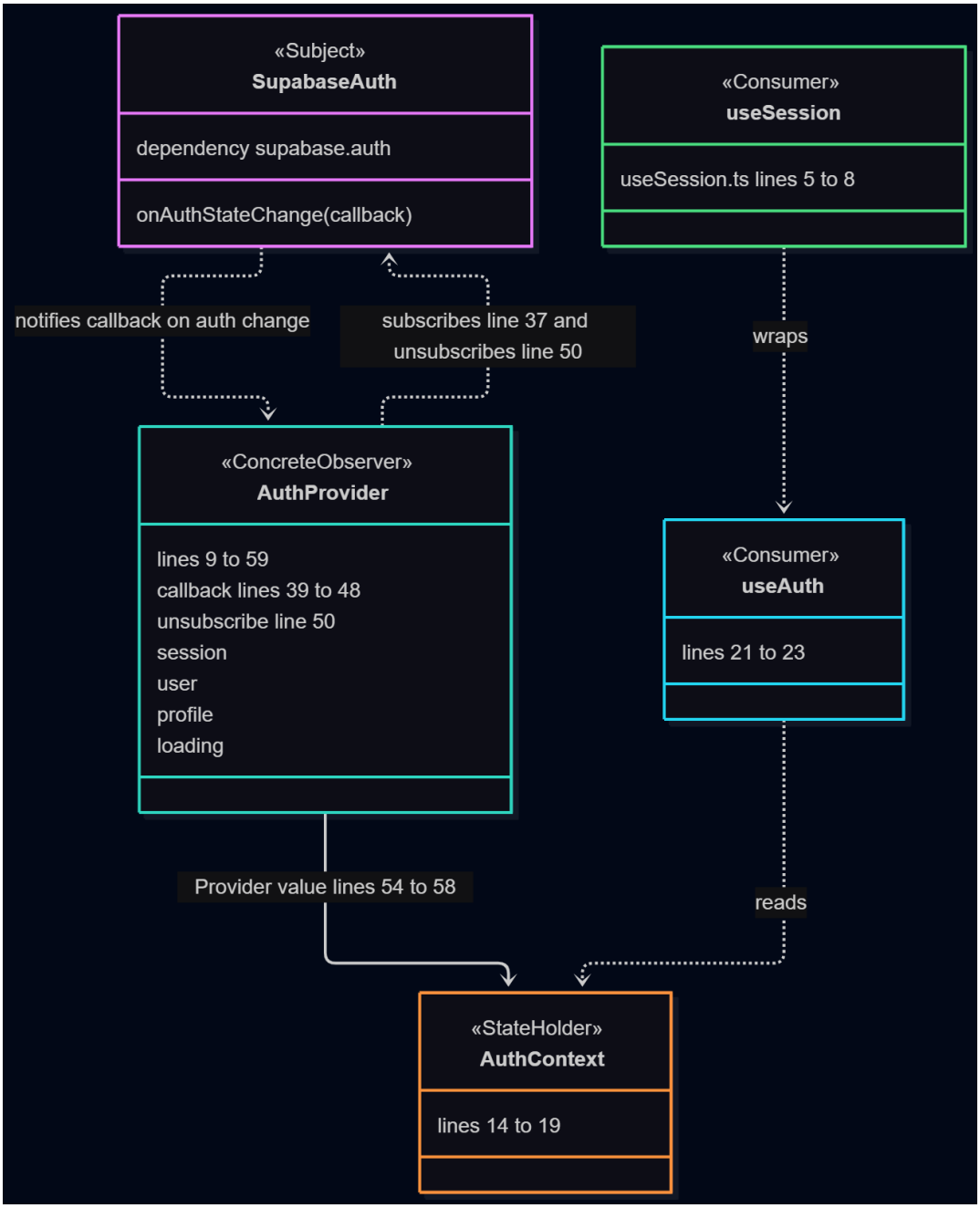


Figure 4: UML for the Observer instance (source: `Part1/uml_observer.mmd`).

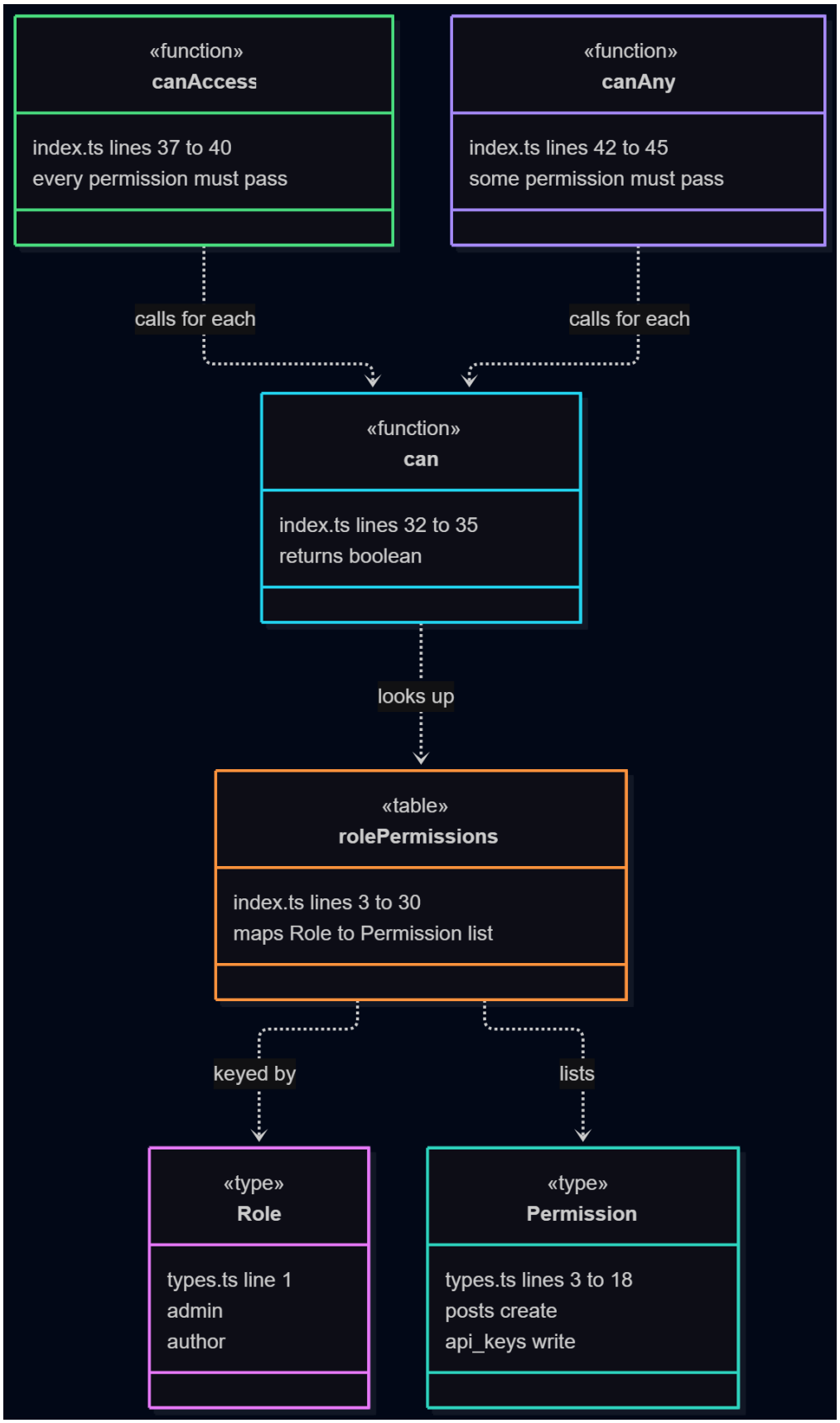


Figure 5: UML for the RBAC instance (source: `Part1/uml_rbac.mmd`).

Verification: Confirmed as an architecture pattern. Reviewers: team B14. Method: source reading; `__tests__/lib/permissions.test.ts` (29 tests) passes in the baseline run.

2.4 Rejected and uncertain candidates

Uncertain (1). `proxy.ts` guard pipeline (lines 4–117): sequential guard checks in one function. It resembles Chain of Responsibility, but there is no handler interface, no successor links and no separate handlers, so roles cannot be mapped without inventing classes. Classified as a Next.js framework idiom.

Rejected (7).

#	Candidate	Reason
R1	Supabase client creators as Factory Method	Four standalone functions chosen by import path; no creator hierarchy or polymorphic selection; `createStaticClient` can return null, so they share no contract
R2	User-confirmed notifications as Observer	`route.ts`:67–74 calls two functions in `Promise.all`, each with its own catch; no subscriber list or notifier interface
R3	`sendNewsletterEmail`	Single function wrapping Resend; one participant
R4	Publication validation as State or Specification	`status` is a plain string; one function with sequential checks, no rule objects
R5	Repeated provider `if` chains	Strategy is not applied; branching duplicated (Part 2 refactoring target)
R6	Helper functions	`buildSystemPrompt`, `resolveGeminiModel`, `isRateLimitError`, `escapeHtml`, `sanitizeSubject`: plain utilities
R7	React Context as a separate Observer	Folded into the Auth State Observer to avoid double counting

3 Part 2: Evolution and refactoring

Scope and traceability. Part 2 evolves the application from the baseline source snapshot `de5bc25` in three requested areas: related-article recommendations, content moderation, and premium-content access control. The implementation is maintained in the application fork `https://github.com/nivedita-rmsh/nextjs-blog-cms`, branch `part2/content-moderation-paywall`. The latest inspected commit is `7ecfa65`. The report's baseline copy in `B14/code/` remains the Part 1 snapshot; it is not a copy of the evolved application branch. The Part 1 pattern inventory is tied to commit `90ec6565877053ee0e93e9f09dcc7c7213aa4832`, while the Part 2 feature work starts from the later application commit `de5bc25`. These snapshots must not be conflated when describing the starting state.

Change	Requirement	Evidence commit	Main implementation areas
CH01	Article recommendations	`68cd127`	`features/posts/recommendations.ts`; `features/posts/components/RelatedPosts.tsx`; public article page
CH02	Content moderation	`376bc05`–`33ca044`	Moderation state, admin review page and actions, public queries, REST publication paths, Supabase migration
CH03	Premium-content access	`a7d6990`–`7ecfa65`	Premium entitlement helper, editor toggle, protected body storage, public article page and Supabase migration

The changes are cumulative on the feature branch. The short commit IDs above identify the first or final relevant commits; the complete history is available in the application repository. Code paths and line ranges below refer to that branch, rather than the Part 1 baseline.

CH01: Related-article recommendations

Requirement. Show a small set of useful related articles on an article detail page, based on existing metadata, without recommending the current article, drafts, unapproved posts, or unrelated content.

Implementation. `rankRelatedPosts()` in `features/posts/recommendations.ts` (lines 7–49) ranks candidates by shared tags and category. Each shared tag contributes two points and a shared category contributes one point; candidates with no positive score are removed. Ties are resolved by publication recency, and the result is limited to three posts by default. The function excludes the current article, drafts and unrelated candidates. The later CH02 moderation change adds the additional approval-state filter to recommendation candidates in the cumulative branch. The async server component `RelatedPosts` in `features/posts/components/RelatedPosts.tsx` (lines 6–23) loads published posts, applies the ranker and renders the results when any remain. The public detail page imports and renders the component at `app/(public)/blog/[slug]/page.tsx:14,143`.

Pattern effect. No GoF pattern is claimed as newly introduced. The ranker is a deterministic domain helper, not a Strategy instance: there is no family of interchangeable ranking algorithms selected through a common contract. The existing LLM Strategy, Auth State Observer and RBAC Permission Table are preserved. The added query/component collaboration is recorded as a feature-level change, not counted as another design-pattern instance.

Verification. The focused file `__tests__/features/posts/recommendations.test.ts` contains four cases covering tag-versus-category ranking, exclusion of the current article and drafts, result limiting/relevance, and empty results. The recorded focused run showed all 4 recommendation tests passing. During the same pre-commit attempt, a separate author-byline test file reported two failures; those failures were not established as pre-existing, so the overall hook result must not be described as fully passing. The commit was created with hooks bypassed after the hook failure. A full passing post-change suite has not been recorded for this snapshot.

CH02: Content moderation before publication

Requirement. Require administrator review before new publication requests become public, apply that rule to the normal editor and supported REST publication paths, and keep pending or rejected content out of public views.

Implementation. The `moderation_status` column supports `not_submitted`, `pending`, `approved` and `rejected`. The migration `supabase/migrations/20261012100000_content_moderation.sql` (lines 1–117) backfills existing published posts as approved, defines the public-read policies, and updates the atomic save function so a publication request is stored as a draft pending review. Edits

to a published article return it to the queue. Public post queries, category/tag listings, sitemap entries, recommendation candidates and the relevant REST routes filter or enforce the approval state; examples include `features/posts/queries.ts` (lines 30, 48, 96, 117 and 148), `app/api/posts/create/route.ts` (lines 29–33, 75–91 and 116–119), and `app/api/posts/[id]/route.ts` (lines 138–140 and 201–208).

Administrators see `/dashboard/admin/moderation`, implemented in `app/(dashboard)/dashboard/admin/moderation/page.tsx` (lines 8–39), with the review controls in `components/dashboard/ModerationQueue.tsx` (lines 20–53). The server actions in `features/posts/moderation-actions.ts` (lines 12–39 and 42–62) perform admin checks, revalidate publication fields, compare the row's update timestamp to avoid acting on stale content, and then approve/publish or reject/return the post to draft. The editor shows a pending moderation message instead of claiming publication succeeded.

Pattern effect. The original RBAC Permission Table remains present; its three baseline participants are still the same. The moderation feature adds admin-role checks and database row-level security (RLS) at new enforcement points. These controls are not a new GoF pattern, and the new explicit admin checks are not presented as a complete centralization of authorization through `rolePermissions`. The existing Strategy and Observer instances are preserved.

Verification status. Five unit cases were added in `__tests__/features/posts/moderation.test.ts` for state transitions and public visibility, and existing post-action and REST tests were adjusted. These focused tests were added but their run result had not been captured at the time of this report update. The migration has not yet been applied and verified against a running Supabase database. TypeScript checking passed on the cumulative feature branch, but this is not a substitute for running the tests or the migration.

CH03: Premium-content access control

Requirement. Allow an author to mark an article premium, show a public preview to readers without access, and disclose the full body only to the author, an administrator or a user with an active premium entitlement. Newsletter subscription is not treated as proof of paid access.

Implementation. The editor schema and `PostEditor` add an `is_premium` flag and checkbox (`components/dashboard/PostEditor.tsx`: 38, 122 and 482–485). The migration `supabase/migrations/20261012103000_premium_content_access.sql` (lines 1–170) adds the flag to `posts`, creates `premium_post_contents` for full premium bodies and `premium_subscriptions` for access entitlements, and adds RLS policies for approved premium content and active entitlements. The atomic persistence function stores a premium post's full body in the protected table and leaves `posts.content` empty for that post; when a post is changed back to free, the full body is restored to the normal field and the premium-body row is removed.

`hasPremiumAccess()` and `getPremiumPostBody()` in `features/premium/access.ts` (lines 5–28) check sign-in, admin/author access, entitlement status and optional expiry before retrieving the protected body. The public article page (`app/(public)/blog/[slug]/page.tsx`, lines 58–60 and 119–128) renders the full content only when access is confirmed; otherwise it renders an excerpt and sign-in call to action. The REST detail endpoint remains API-key protected and scopes a post read to the API-key owner's posts; it is used for author operations, not anonymous public paywall access.

Scope limitation. This change implements entitlement-based access control, not a payment or billing system. An active row in `premium_subscriptions` must be provisioned through an administrative or separate trusted process; the application does not process payments, create paid subscriptions, or handle renewals.

Pattern effect. The existing Strategy, Observer and RBAC instances remain present. Premium entitlement checks and protected-content persistence are additional security/domain rules, not a

demonstrated GoF pattern. No new pattern is counted based solely on the helper's name or on the presence of a subscription table.

Verification status. Six test cases were added in `__tests__/features/premium/access.test.ts` for anonymous denial, active and expired entitlements, revoked entitlements, administrator access, author access to a protected body and denial before protected content is queried. The test file was added but its run result had not been captured at the time of this report update. The cumulative branch's TypeScript command `pnpmexectsc--noEmit` completed with exit code 0. The new Supabase migrations and premium-content flows still require database/runtime verification; no full-suite passing result is claimed for this branch.

Pattern evolution summary

The baseline inventory contains three confirmed instances: LLM Provider Streaming Strategy, Auth State Observer and RBAC Permission Table. Source inspection of the feature branch does not establish a fourth GoF instance, so these counts remain three across the snapshots below. The table describes the three confirmed Part 1 instances only; it does not count every feature module.

Snapshot	Commit	Strategy	Observer	RBAC	Total confirmed instances
Baseline	`de5bc25`	1	1	1	3
After CH01 recommendations	`68cd127`	1	1	1	3
After CH02 moderation	`33ca044`	1	1	1	3
After CH03 premium access	`7ecfa65`	1	1	1	3

Status classification. The LLM Provider Streaming Strategy and Auth State Observer are preserved through these changes. The RBAC Permission Table is also preserved as an instance: `rolePermissions`, `can`, `canAccess` and `canAny` are not replaced by a new GoF structure. Moderation and premium access extend the application's authorization surface through explicit checks and RLS, but are not recorded as separate RBAC-pattern instances without evidence of a new, centralized role/permission collaboration. The recommendation ranker likewise remains a normal helper rather than a formal Strategy. The feature snapshots and counts must be reflected in the final machine-readable `pattern_evolution.csv` and `Pattern_count.csv`; the current CSVs need a final consistency check against these snapshots before submission.

Supporting refactoring analysis

In addition to the three required feature changes, three refactoring experiments were recorded on separate application branches. These are supplementary refactoring evidence and are not the CH01–CH03 feature changes above.

Refactoring evidence	Code smell and technique	Pattern/verification note
`e7dfd1c`	Repeated model-name inference; extract `inferProviderFromModel()` and use it in `generateBlogPost()`.	Streaming Strategy collaboration is preserved; this helper does not establish a new Strategy instance. The implementation record reports 23 focused tests and 1,025 tests across 88 files, while a subsequently stored focused log shows 20 tests; those records must be reconciled.
`7be15fb`	Inline title and readable-body checks in publication validation; extract `validatePublicationTitle()` and `validatePublicationContent()`.	No new GoF pattern claimed. The implementation record reports 30 focused tests before and after and 1,022 tests across 88 files.
`6ce7157`	Duplicate JSON cleanup, parsing and error handling in `generateBlogPost()` and `generateBlogPostHeadless()`; extract `parseGeneratedPostJson()`.	No new GoF pattern claimed. The implementation record reports 1,022 tests across 88 files; the focused log's exact branch context needs confirmation.

The refactoring commits have separate branch histories and are not automatically part of the feature branch. Supporting records reside in the existing `changes/Change1`, `changes/Change2` and `changes/Change3` folders. Those folder names originated with the refactoring experiments and should be renamed or reorganized so they cannot be confused with the required feature records CH01–CH03.

LOC and remaining verification

A consistent before/after source-lines-of-code (LOC) metric has not yet been captured using an explicit counting rule across the same paths. This report does not invent a LOC delta. Before final submission, count production source lines using the same tool and rule at each stated snapshot and record the commands and outputs. Also run the focused tests for CH02 and CH03, investigate the two reported author-byline test failures alongside CH01, run the full test suite on the cumulative feature branch, and apply/verify both Supabase migrations against a test database. The baseline report's lack of a local Docker/Supabase setup explains why runtime verification was unavailable at that point; it does not prove the new database changes work.

4 Part 3: Cross-language comparison

Objective and scope. Part 3 compares small, equivalent implementations of two confirmed patterns from Part 1: *LLM Provider Streaming Strategy* and *Auth State Observer*. The RBAC Permission Table is intentionally excluded from this working comparison to follow the team's request to use two patterns. The examples isolate pattern behaviour for controlled testing; they are not complete ports of the Next.js application and do not call real LLM services or Supabase.

Language scope. The repository contains Python and JavaScript examples by Govind and Java and C++ examples by Soorya for both selected patterns. Python and JavaScript share 26 saved input/output cases. Java and C++ share a different, smaller specification with four cases per pattern. All four languages are present, but one common four-language behavioral check has not

yet been run.

Selected patterns and source mapping

Strategy — LLM Provider Streaming Strategy. Part 1 confirms this instance in `code/features/ai-assistant/llmService.ts`. The concrete provider functions are `streamClaude`, `streamOpenAI` and `streamGemini`; `sendMessage` selects a provider from `params.provider` and delegates to the corresponding function. The original TypeScript code has an implicit function contract and a hard-coded conditional chain rather than a separately declared strategy interface. Govind's Python and JavaScript examples keep function-based strategies, using `run` to select a provider-specific extractor for saved sample events. For comparison, the Java version defines an `LlmStrategy` interface and concrete strategy classes, while the C++ version uses an abstract base class and concrete implementations. The Java and C++ examples return fixed chunks. None of the four examples calls a provider service. These are teaching adaptations, not claims that the TypeScript code has the same class structure.

Observer — Auth State Observer. Part 1 confirms this instance in `code/features/auth/context/AuthProvider.tsx`, `AuthContext.tsx` and `code/features/auth/hooks/useSession.ts`. The original subject is `supabase.auth`, supplied by the Supabase dependency; `AuthProvider` registers the callback for `onAuthStateChange`, updates session state and unsubscribes in effect cleanup. Govind's Python and JavaScript examples use an in-memory `AuthSubject` and an `AuthStateObserver` that records user IDs. The Java and C++ examples model an event source, listeners, event delivery and unsubscribe behaviour. None connects to Supabase.

Relationship to Part 2

The uploaded Part 2 records describe three changes: CH01 related-article recommendations, CH02 content moderation, and CH03 premium-content access control. They also describe supplementary refactoring experiments involving provider inference, publication validation and generated-post JSON parsing. These feature changes do not establish a new GoF pattern for this comparison. CH01 and the provider-inference refactor are related to the Strategy area, but the Part 2 implementation notes explicitly say not every provider-selection branch was refactored.

Source-snapshot limitation. The copied `B14/code/features/ai-assistant/llmService.ts` in the uploaded project still contains the baseline provider-selection branches and does not contain the helper names described in the refactoring notes. Consequently, the snippets below compare the verified Part 1 baseline behaviour and do not claim to benchmark a merged, post-refactoring source snapshot. The baseline and Part 2 feature branch commits are recorded separately in the Part 2 section and must not be conflated.

Common test specification

Java and C++ use the same cases from `Part3/test-cases/common_test_cases.csv`:

Test ID	Pattern and input case	Expected behaviour
S01	Strategy: provider `claude`, prompt `hello`	Return chunks `[claude:,hello]` in order.
S02	Strategy: provider `openai`, prompt `hello`	Return chunks `[openai:,hello]` in order.
S03	Strategy: provider `gemini`, prompt `summary`	Return chunks `[gemini:,summary]` in order.
S04	Strategy: unknown provider	Reject with `unsupportedprovider:unknown`.
O01	Observer: sign-in event	Active observer receives `signed\_in:session-1`.
O02	Observer: sign-out event	Active observer receives `signed\_out:none`.
O03	Observer: unsubscribe then publish	Unsubscribed observer receives no subsequent event.
O04	Observer: two active observers	Both active observers receive the same sign-in event.

Java and C++ test results. The recorded output in `Part3/evidence/test_results.txt` shows:

Pattern	Java	C++
Strategy	4/4 passed	4/4 passed
Observer	4/4 passed	4/4 passed

The test runners are `Part3/strategy/java/StrategyTest.java`, `Part3/strategy/cpp/test_strategy.cpp`, `Part3/observer/java/ObserverTest.java` and `Part3/observer/cpp/test_observer.cpp`.

Python and JavaScript test results. Govind's two languages use the same JSON cases in `Part3/llm_provider_streaming/fixtures.json` and `Part3/auth_state_observer/fixtures.json`. Each test runner checks the expected result and that its input was not changed. A separate process check invokes both programs for each saved input:

Pattern	Python	JavaScript	Process check
Strategy	11/11 passed	11/11 passed	11/11 matched
Observer	15/15 passed	15/15 passed	15/15 matched

The combined process check passed 26/26. Its script is `Part3/verify_cross_language.py`; the saved output and four language-specific logs are in `Part3/evidence/tests/`. Production snippets and run commands are in `Part3/README.md`. The Python/JavaScript JSON contract differs from Java/C++ direct provider/prompt and event/session calls. The two pass counts do not establish one shared four-language test result.

Measurement method

Soorya measured the Java and C++ examples as follows:

- **Source LOC:** Counts production files only, excluding blank lines, comments, preprocessor guards and test code.
- **Cyclomatic complexity:** A function-by-function manual McCabe audit uses $M = 1 + D$. Each `if` and loop is one decision; each non-default `switch` case is one decision. Abstract declarations and library internals are excluded. The audit is documented in `Part3/evidence/complexity_audit.csv` and `Part3/evidence/complexity_audit.md`; it is not the output of an external static-analysis tool.

- **Timing:** Compilation is performed before timing; one warm-up run is followed by five measured process launches on the same environment. The median includes process startup and the test harness but excludes compilation. Raw runs are in `Part3/evidence/timing_raw.csv`; toolchain versions are in `Part3/evidence/environment.txt`.

Govind used Lizard 1.24.1 for production source lines and maximum function complexity in Python and JavaScript. His timing script ran one warm-up and five measured whole-process launches on Windows, including startup and JSON input/output. The raw records are in `Part3/evidence/static_metrics.json` and `Part3/evidence/timings/process_timings.json`. These methods and machines differ, so the two sets of measurements are descriptive rather than a fair language speed ranking.

Results

Pattern	Language	Production LOC	Max function CC	Median process time (ms)
Strategy	Python	53	8	96.870
Observer	Python	74	21	100.344
Strategy	JavaScript	47	9	57.485
Observer	JavaScript	62	20	57.960
Strategy	Java	33	4	36.116
Observer	Java	24	2	42.995
Strategy	C++	35	4	1.115
Observer	C++	30	2	1.070

In Python and JavaScript, most of the Observer maximum function complexity comes from validation in `run`, not notification itself. For Java and C++, the maximum-complexity functions are `LlmContext.select()` (Java Strategy), `LlmContext::stream()` (C++ Strategy), and `AuthEventSource.publish()` for Observer. Their manual decision counts are in `Part3/evidence/complexity_audit.csv`.

The two pairs were measured on different machines and with different process boundaries. These numbers cannot rank the languages. The snippets are small, so startup and the test harness affect timing. The C++ Observer uses non-owning pointers in this teaching example; production C++ would require a clear lifetime and ownership policy.

Interpretation and limitations

Python and JavaScript express Strategy as functions selected by a map. Java uses an interface and concrete classes; C++ uses an abstract base class and virtual methods. All four Observer examples hold listeners, notify active ones and support unsubscribe. The original app instead uses Supabase's subscription API and React Context.

Govind's four records are in `data/cross_language.csv`; Soorya's four separate records remain in `Part3/cross_language.csv`. The team still needs one input/output specification, a shared four-language test run and a consistent measurement boundary before combining them. The Part 3 LLM record is partial: the prompt and tests are saved, but raw output and reliable model/date details were not archived. The Part 2 feature branch also needs its own remaining verification; no result here claims to measure its post-change application code.

5 LLM records and human verification

Run and model. Part 1 pattern analysis used an LLM chat assistant on 2026-10-10. The displayed model name, exact prompts, raw outputs and the evaluation of each analysis are recorded in `Part1/`

`llm.csv`, `Part1/Part1_prompt.txt` and `Part1/llm/`.

Claims and evaluation. Every pattern claim was checked against the numbered source listings in `Part1/evidence/`. Corrections made after verification: the line number given for `getUser()` in `proxy.ts` was wrong and the true line is 34; a claim that the Users page uses `can()` and the server Supabase client was unsupported, since the page calls `requirePermission` and `getAllUsers`, so it was removed; and an initial Observer guess for the user-confirmed notification fan-out was rejected after reading `route.ts`. No generated code was used in Part 1.

Part 2 records. The intended feature scope is preserved in `Part2/Part2_prompt.txt`; source commits and test logs are referenced in this section and in the Part 2 change records. The raw Part 2 model-response transcripts were not separately archived as reproducible files when this report was updated, so this report does not invent model output or treat proposed prompts as completed model runs. The Part 2 LLM-use record and verification status should be checked against the actual saved chat transcripts before final submission.

Part 3 records. Govind's saved task prompt is in `Part3/llm/govind_prompt.txt`. Source and test checks are described in `Part3/llm/verification.md`. The row in `Part3/llm/llm.csv` marks the full raw assistant output, displayed model name and interaction date unavailable. It does not claim a complete transcript or completed human review. Those fields should only be updated from verifiable saved history.

6 Findings, contributions and references

Findings. At the baseline commit, three pattern instances were confirmed (Strategy, Observer, RBAC), one candidate is uncertain and seven were rejected. The application mostly relies on framework idioms and plain functions rather than classical GoF structures. The Strategy is function-based with an `if`-chain selector and no declared interface. The Observer subject comes from a dependency. RBAC is an architectural pattern. Of the eight areas in the brief, articles, authors, categories, comments, authentication and newsletter subscription exist; moderation exists only as comment deletion, and recommendations and a paywall were not found. Limitations: the app was not run (no Docker), and behavior was verified from source and the unit-test suite only.

Contributions.

Roll number	Name	Batch	Contribution
AM.SC.U4AIE23121	Keziah Suzaine George	AIE	Part 1 pattern analysis
AM.SC.U4AIE23139	Nivedita Ramesh	AIE	Part 2 change records
AM.SC.U4CSE23360	Soorya Krishna	CSE	Part 3 Java and C++ examples
AM.SC.U4CSE23309	B R Govind	CSE	Part 3 Python and JavaScript examples

References.

1. E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
2. R. Sandhu, E. Coyne, H. Feinstein, C. Youman, "Role-Based Access Control Models," *IEEE Computer* 29(2), 1996.
3. Next.js Blog CMS, `https://github.com/thepracticalengineer/nextjs-blog-cms`, commit 90ec656, MIT license.
4. Next.js documentation, `https://nextjs.org/docs`.
5. Supabase documentation, `https://supabase.com/docs`.
6. Mermaid, `https://mermaid.js.org`.

7 Team folder and datasets

The `B14/` folder contains the baseline application in `code/`, its baseline test evidence in `test_result/`, pattern discovery in `Part1/`, change records in `Part2/`, and the cross-language examples in `Part3/`. The central cross-language CSV is `data/cross_language.csv`; the separate Java/C++ rows are still in `Part3/cross_language.csv` until one common comparison is verified.